Table C.1 Built-in Django template tags *(continued)*

Tag	Description
`{% templatetag value %}`	This outputs one of the special characters normally reserved for the template tag language. It accepts an argument naming the special character, which is either openblock ({%), closeblock (%}), openvariable ({{), closevariable (}}), openbrace ({), closebrace (}), opencomment ({#), or closecomment (#}).
`{% url 'name' arg1 arg2 %}`	This looks up the named URL and returns an absolute path reference. Arguments after the name are optional and are passed to the URL look-up for URLs containing values.

`{% verbatim %}`
`  {% if count > 50 %}`
`{% endverbatim %}`
	The contents of a `{% verbatim %}` tag are rendered as is. This is typically used to include JavaScript templating syntax that uses similar characters as Django's template language. An optional name can be provided in both the open and close tag, allowing you to nest `{% verbatim %}` tags using pairs of names.

`{% widthratio value max max_width %}`
or
`{% widthratio value max max_width as my_ration %}`
	This calculates a value for a width based on the ratio of `value` to `max`. For example, if `value` is 50, `max` is 100, and `max_value` is 60, the result would be 30 (50 ÷ 100 * 60). Also accepts an "as" argument to put the result into a variable instead.

`{% with name="Mike" %}` `{% endwith %}`	This defines a context variable for use within the block.

C.1.1 Loadable tags

Django's built-in tags are available without using a `{% load %}` tag. Django also ships with other tags that are inside modules that need to be loaded to be used.

Table C.2 Loadable Django tags

Tag	Module	Description
`{% static "images/photo.jpg" %}`	static	This returns a link to the named static file.
`{% get_static_prefix %}`	static	This returns the relative path portion of a static URL.
`{% get_media_prefix %}`	static	This returns the relative path portion of a media URL.

`{% translate "Text for translation" %}`
or
`{% translate "Text for translation" as my_translation %}`
	i18n	This returns the translated text equivalent looked up within the translation files. It supports both string constants and named variables as well as an `as` clause to have the result put into a variable instead of being rendered.